

4. App Engine: Deploy a web application on App Engine with automatic scaling enabled

To deploy a web application on Google App Engine with automatic scaling enabled, you need to follow these steps:

Prerequisites:

1. **Google Cloud account:** Ensure you have a Google Cloud account and access to the Google Cloud Console.

Steps:

Step 1: Set up your project directory

Create a new directory for your web application or navigate to your existing project directory.

If you're starting a new project, here's an example structure for a simple Python Flask application:

```
your-app/  
├── app.yaml  
├── main.py  
└── requirements.txt
```

- `main.py`: Your application code (Flask app for example).
- `requirements.txt`: List of dependencies for your project (e.g., Flask).
- `app.yaml`: Configuration file for App Engine.

Step 2: Write your web application code

Here's a simple example of a Python Flask application (`main.py`):

```
from flask import Flask  
app = Flask(__name__)
```

```
@app.route('/')
def hello():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
    #app.run(host='0.0.0.0', port=8080) #use this or app.run()
```

Step 3: Create a `requirements.txt` file

In the `requirements.txt` file, list your application dependencies. For the example above, it would look like:

```
Flask==2.1.0
```

Step 4: Create an `app.yaml` file

The `app.yaml` file is crucial for configuring your App Engine deployment, including automatic scaling. A basic `app.yaml` for automatic scaling looks like this:

```
runtime: python39
automatic_scaling:
  min_instances: 0
  max_instances: 5
  target_cpu_utilization: 0.65
```

// Alternative script for the file `app.yaml` shown below //

```
runtime: python39 # Specify the Python version
instance_class: F2 # This is an optional field to specify
```

```
env_variables:
  # Add any environment variables if necessary
automatic_scaling:
  target_cpu_utilization: 0.65 # Automatically scale based
on CPU usage

  min_instances: 1 # Minimum number of instances
  max_instances: 10 # Maximum number of instances

// end of script //
```

The `automatic_scaling` section is what enables the scaling behavior, and you can adjust the parameters like `min_instances`, `max_instances`, and `target_cpu_utilization` to suit your needs.

Step 5: Deploy the application

Before deploying, authenticate your Google Cloud SDK:

```
gcloud auth login
```

Then, initialize your project:

```
gcloud init
```

Now, deploy your app to App Engine:

```
gcloud app deploy // works only with billing
```

This will upload your application and deploy it to App Engine, where it will be automatically scaled based on demand.

Step 6: Access your application

Once the deployment is complete, you'll see a URL where your app is hosted. You can access your web application via this URL:

```
gcloud app browse // works only with billing
```

This will open your deployed application in your default web browser.

//Go To Note Section to execute python file manually and to run the flask application

Step 7: Monitor and manage your app

You can monitor your application's performance and scaling through the Google Cloud Console. The automatic scaling feature ensures that as your traffic increases, the app scales up and down automatically based on the usage.

Additional Tips:

- **Scaling parameters:** You can tweak the `automatic_scaling` parameters in `app.yaml` to change how scaling behaves (e.g., based on memory usage or request count).
- **Log management:** Use Google Cloud's Stackdriver Logging to monitor logs and manage error reporting.
- **More advanced configurations:** You can specify more advanced features in `app.yaml`, such as `handlers`, `env_variables`, and custom domains.

Please Note:

Without Billing, app deployment on cloud won't be possible. However, the app can be deployed on localhost.

Execute the following command to see the output of web app on local host through browser:

On terminal, execute the python file as

`python3 main.py`

Afterwards click on the link displayed: 127.0.0.1 to see the output

Screenshots:

```
raghavendra_ih@cloudshell:~ (global-bridge-450512-j6) $ mkdir myapp
raghavendra_ih@cloudshell:~ (global-bridge-450512-j6) $ cd myapp
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6) $ vi main.py
```

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6) $ vi requirements.txt
```

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6) $ vi app.yaml
```

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6) $ gcloud auth login
```

```
You are already authenticated with gcloud when running
inside the Cloud Shell and so do not need to run this
command. Do you wish to proceed anyway?
```

```
Do you want to continue (Y/n)? y
```

```
Go to the following link in your browser, and complete the sign-in prompts:
```

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6) $ gcloud init
Welcome! This command will take you through the configuration of gcloud.
```

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6)$ gcloud app deploy
Services to deploy:

descriptor:          [/home/raghavendra_ih/myapp/app.yaml]
source:              [/home/raghavendra_ih/myapp]
target project:      [global-bridge-450512-j6]
target service:      [default]
target version:      [20250329t045151]
target url:          [https://global-bridge-450512-j6.el.r.appspot.com]
target service account: [global-bridge-450512-j6@appspot.gserviceaccount.com]

Do you want to continue (Y/n)? y
```

```
Beginning deployment of service [default]...
Created .gcloudignore file. See `gcloud topic gcloudignore` for details.
Uploading 3 files to Google Cloud Storage
33%
67%
100%
100%
File upload done.
Updating service [default]...done.
Setting traffic split for service [default]...done.
Deployed service [default] to [https://global-bridge-450512-j6.el.r.appspot.com]

You can stream logs from the command line by running:
  $ gcloud app logs tail -s default

To view your application in the web browser run:
  $ gcloud app browse
```

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6)$ gcloud app browse
Did not detect your browser. Go to this link to view your app:
https://global-bridge-450512-j6.el.r.appspot.com
```

To check the app running on localhost:

```
raghavendra_ih@cloudshell:~/myapp (global-bridge-450512-j6)$ python main.py
* Serving Flask app 'main'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment
. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Hello, World!